

关于 RAG 优化的思考记录

- 背景
- 现状
- RAG 优化
- RAG 评估体系
 - 常见的评估指标
 - 检索可观测性
 - 评测集模板
 - 评估能力能否平台化
- 问题解答

背景

近期在做 FE 结合 AI 的应用落地，比如 AI 结合私有组件库生成 UI 代码， AI 结合业务知识生成业务场景的测试用例， 这些场景都免不了用到知识库能力。在进入深水区的时候，就必然要遇到 RAG 如何选择，如何优化的问题。

最早使用知识库是在 FE AI 翻译的项目里，因为业务上（供应链领域）有比较多专业术语的翻译，最开始通过提示词直接给到 LLM，导致 token 消耗大，后来通过接入 smart 知识库，解决了这个问题。因为翻译词条的结构都比较简单，而且都是专业术语，单个词条文本也不至于太长，在数据预处理和检索上实际都没遇到太大问题，也没有深入去调研 RAG 。但是走到生成代码和生成测试用例时， RAG 检索的准确性、返回的完整性就对 AI 生成结果产生了巨大的影响。

注：本文不讨论微调，虽然微调也可以实现私域场景的知识生成，基于成本、知识更新频率、所处阶段等考虑，暂时不考虑微调，只讨论 RAG 的方式。

现状

目前几个项目对知识库的使用方式大致如下：

项目	RAG	考虑
----	-----	----

AI 翻译	smart platform	翻译词条结构简单，直接接入公司平台，简单快捷
AI 生成 UI 代码	alpha platform	主要是生码需要 workflow 能力，alpha 平台对 workflow 的支持更好，所以顺带使用了 alpha 平台提供的知识库能力
AI 生成自测用例	自己用 nodejs 实现，方案	不做向量化的考虑详细分析： link ，简单来说两点： (1) 数据量不大 (2) POC 阶段，简单实现已可覆盖 80% 以上的场景

其他团队项目：	smart platform	知识库数据量大
FE 组件库助手、FE 答疑机器人	alpha platform	接入公司平台，简单快捷

基于现状，大体把 RAG 的实现分为两个方式

(1) 公司平台

(2) 自研

自研的方式可以做更多精细化的策略优化，比如在 AI 生成代码这个场景，对于代码可以用专用的向量化模型（CodeBERT、UniXcoder）等，但是自研如果要做精细化，无疑也是投入成本更高的。考虑成本，我们第一期的调研还是围绕怎么基于公司已有平台的知识库和 RAG 做优化策略。

我们目前在实践中，关于 RAG 频繁遇到几个问题：

1. 【AI 自测用例生成项目】：知识库里有相关的用例，但是检索的时候就是召不回怎么办？
2. 【PRD2Code 项目】：如果生码效果差怎么区分是物料的问题还是生成的问题？
3. 【PRD2Code 项目】：为什么我总结的优化文档和技术方案总是被挑战？

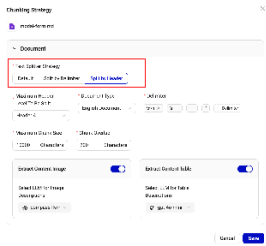
接下来会引入一些知识，最后来解答这三个问题。

RAG 优化

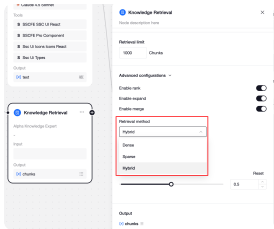
关于 RAG 的优化策略很多，附录放了一些科普资料，为方便阅读，在这里做一个简单的结构化总结：

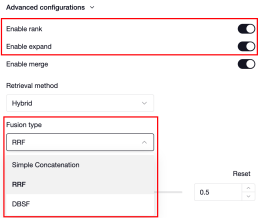
阶段	策略	思路概述	适用场景	平台支持情况
数据预处理	简单分片	按特定字符长度分片，相邻块之间允许存在重复字符		Smart 支持一些特定的分片切割方式配置

阶段	动态语义分片	按段落、语义拆分，超阈值再字符拆分，避免语义断裂	
	锚点导向分片	匹配“条款 X”“步骤 X”等锚点标识拆分，保障结构化文档内容完整性	
	分片增强适配	动态调整分片重叠率，补充元数据标注，同时校验分片质量减少噪声	
	分层索引构建	建立摘要级和文档块级两级索引，先检索摘要再定位具体文档块，适配海量数据	
查询阶段	多查询重写	将原始问题扩写为 3 – 5 个相关子问题，并行检索后合并去重	平台本身不支持，但可以通过其他手段支持



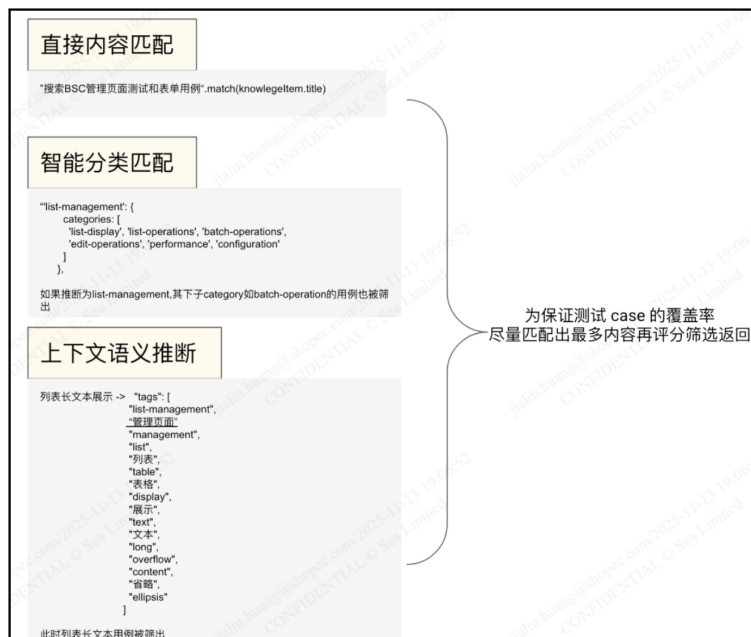
	复杂问题分解	拆解多维度问题为独立子问题，分别检索后汇总答案	
	Step – Back 策略	生成原始问题对应的本质抽象问题，辅助模型理解核心原理	
	HyDE (假设性文档嵌入)	用 LLM 生成问题的假答案，与原查询共同嵌入检索 弥补原始查询语句模糊的问题	
检索阶段	混合检索	融合稀疏检索（如 BM25）和密集检索（向量搜索），配置权重互补优势	alpha 支持配置稀疏、密集、混合
	多路召回	结合多源检索结果，避免单一检索遗漏高价值文档	黑盒，应该是不支持



	句子窗口检索	以句子为检索单位，附带前后若干句上下文，平衡匹配精度与关联性	
	元数据过滤	检索前先按时间、类别等元数据筛选，减少无关结果干扰	
	多轮检索	常见于 Agentic RAG 等进阶架构，核心是通过多轮迭代检索补充信息、修正方向	平台本身不支持，但可以通过其他手段支持
结果重排与生成阶段	简单重排	人工定义规则、关键词匹配、预设权重得到综合得分，按得分排序	alpha 支持简单的排名、融合方式配置，具体参数不支持配置
	RRF 融合重排	无参数的规则式融合方法，和简单重排的区别主要在于融合多源检索结果，以及融合规则上的差异	
	Cross-Encoder 重排	有参数的深度学习模型，基于模型进行语义上的打分，计算成本高，语义理解能力强	

	提示压缩优化	对检索到的上下文做摘要提取，减少冗余信息，聚焦核心内容		
整体性优化	自反馈机制	收集生成结果的反馈，反向调整检索参数与分片规则		平台本身不支持，但可以通过其他手段支持
	智能查询路由	根据问题类型，将查询导向适配的检索器或索引层		
	Few-Shot 提示优化	提供少量示例，引导模型规范输出格式		
	多模态 RAG	与多模态 LLM 结合，实现跨模态的 RAG		

比如 AI 辅助自测在 POC 阶段所采用的一些 RAG 实现基本上都可以映射到上述表格，“直接内容匹配”就是一种稀疏检索，“上下文语义推断”就是一种句子窗口检索，“相关性评分排序”就是一种预设规则和权重的简单重排。



重排策略：相关性评分排序

根据当前测试用例规范（习惯），得到因子及权重

用例示例：[WorkForce 用例池](#)

多因子综合评分（10个因子，权重不同）：

因子	权重	说明
标题匹配	20	标题包含查询关键词
标签匹配	5/个	标签匹配数量
分类匹配	15	知识分类与查询意图匹配
描述匹配	8	描述包含关键词
优先级	5-15	P0=15, P1=10, P2=5
使用频率	$\log(\text{count}) * 2$	历史采纳次数
内容完整度	0-10	字段完整性评分
更新时间	0-5	最近更新加分
作者权重	0-3	高贡献作者加分
多标签加成	$\sqrt{\text{count}} * 2$	多标签覆盖加成

前面现状中提到，我们还是基于公司平台来做优化。平台 RAG 能力整体来说是个黑盒，虽然可以简单配置，但无法直接修改内部检索与向量化逻辑，那么我们核心可聚焦在**输入侧强化数据适配**、**中间侧补充检索增强**、**输出侧优化生成质量**这三个可控环节，同时通过反馈闭环推动策略迭代。简单说，我们的优化策略应该选择**可控的**和**合适的**。可控的指我们可以通过平台控制的或者我们可以在平台提供的能力之上做叠加优化的。合适的指适合我们场景，且在成本和效果上相对平衡的。

本文旨在分享方法，不涉及项目中的具体策略和效果，这部分后续由各个具体项目各自分享。

RAG 评估体系

借用一句话：Evals are important。AI 应用已经从「概念性验证阶段」，转向「能真正衡量并解决问题」的阶段，在这个阶段，评估和优化是同等重要的。不管是 使用自研 RAG 算法或者切换到公司平台， 还是使用平台上的某个优化参数， 还是叠加某种优化策略， 都需要一个可以量化的评估体系来做决策。在 AI 时代，评估将会替代以往的测试（广义上的测试）。

常见的评估指标

- 检索指标
 - 召回率：检索结果中命中查询语句的数量 / 所有与查询语句相关的数量。指标目的：避免漏检。
 - 准确率：检索结果中与查询语句相关的数量 / 所有检索返回的数量。指标目的：避免误检。
 - F1 指标： $2 * (\text{精确率} * \text{召回率}) / (\text{精确率} + \text{召回率})$ 。指标目的：平衡召回率和准确率
 - MRR：所有查询中，首个相关文档排名的倒数。指标目的：找得快不快

- 生成指标
 - 完整性：生成答案覆盖查询需求的比例。指标目的：避免遗漏信息。 比如在生码场景，可能是（1）要求生成 3 个组件，实际只生成了 2 个组件，完整性为 60%（2）代码的完整度
 - 准确率：生成结果与知识库一致的数量/所有事实数量。指标目的：避免胡编乱造。 比如在生码场景，可能是（1）正确使用知识库里的 prop / 所有使用的 prop （2）代码的准确性
 - 相关性：生成结果与查询意图的匹配程度。是一个范围分数。指标目的：相关性。
 - 合规性：生成结果符合预设的格式。指标目的：按格式输出。
- 性能
 - 耗时
 - Token 消耗
- 主观指标

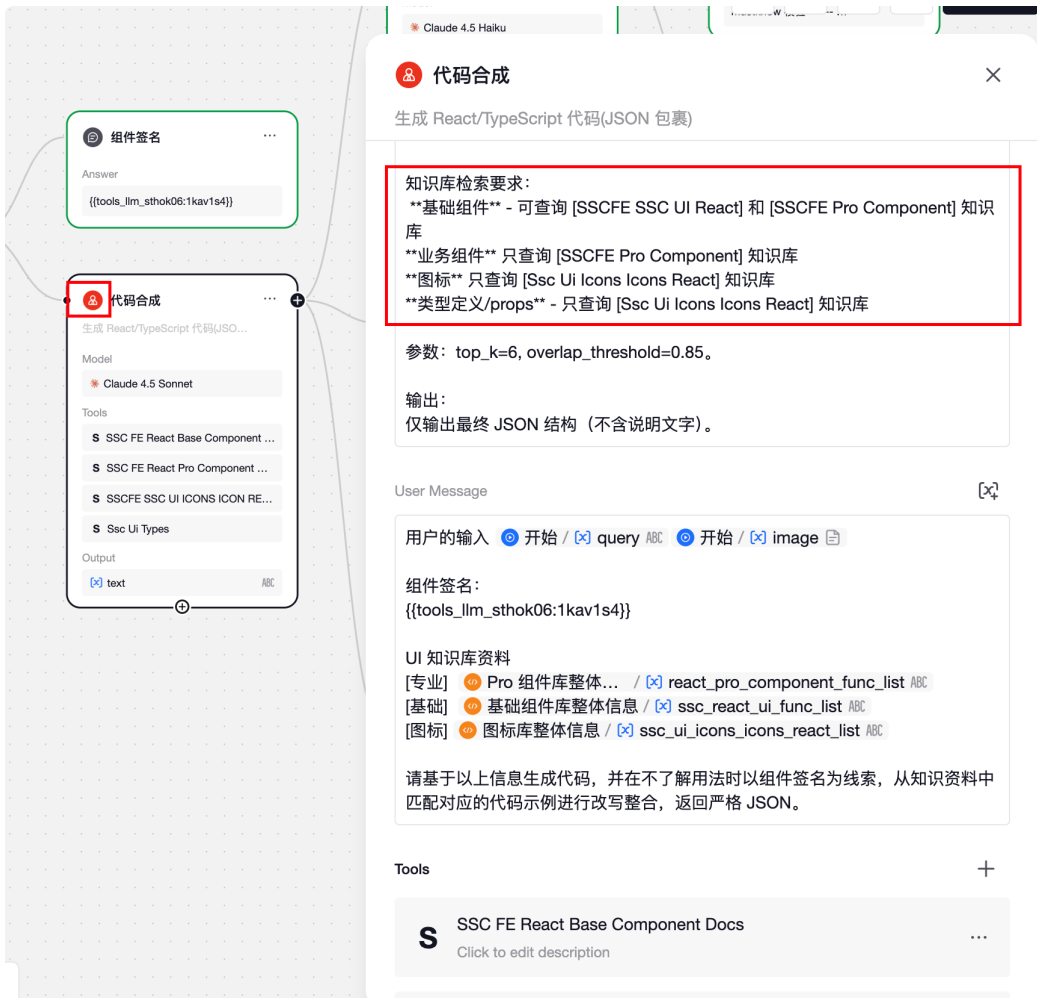
检索可观测性

想要评估 RAG 的效果，对 RAG 的建设是有约束的。就好比在前端项目中，我们想要做一个作业环节的效率优化，如果只有一个整体的耗时，那么是不具备可观测性的，也很难进一步去诊断和制定优化方案，我们要拆分成前端代码执行、扫描耗时、网络延迟、接口调用等多个子步骤，才能让这个作业操作变得可观测，再针对性优化。在不具备可观测性的情况下做 RAG 优化，就是瞎猜，好比在作业效率优化时：先换个接口试试，再删点前端逻辑，又换换 CDN 部署——效率极低，甚至可能改错方向。

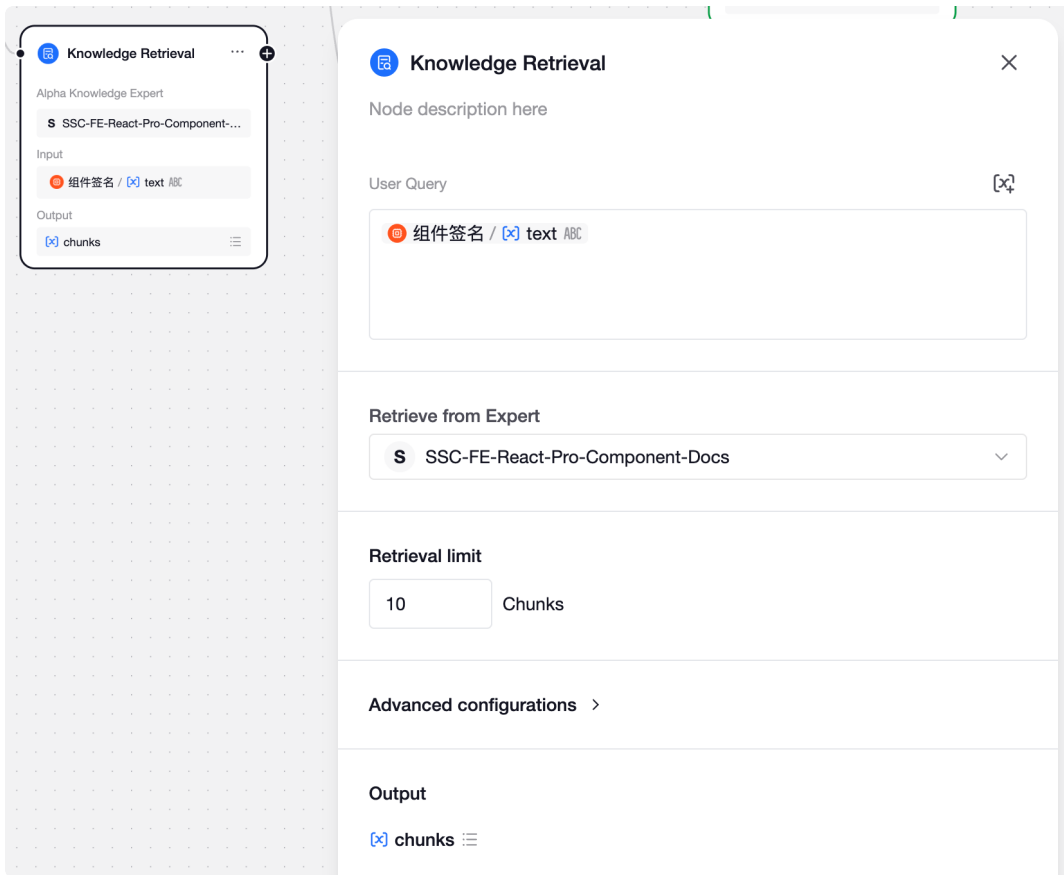
RAG 的可观测性

1.第一种情况，是使用 Alpha 或 Smart 平台搭建的 workflow，对 workflow 的搭建会有一定的约束，要求 workflow 具备检索可观测性。

例如我们当前的 workflow 是这样的：实际上是用一个 LLM 节点把 检索和生成合并为一步了，通过提示词引导 LLM 使用知识库进行检索。



而原生的知识库节点是这个样子的, 可以单独评估这个节点的输入输出。可以在知识库检索节点之后再接一个 LLM 生成节点, 这样可以让单独的知识库节点具备可观测性。



虽然不管是 Alpha 还是 Smart 的知识库都提供了 playground 可供单独调用和测试，但是在 playground 里的参数配置和在 workflow 里是完全不同的，在 playground 里的测试结果不能代表实际结果。

Playground

Test and optimize the expert, trace its thinking process

Question

列表查询

Send

Sources

tabs.md MD425925 ↗

message.md MD425895 ↗

modal.md MD441300 ↗

button.md MD425818 ↗

steps.md MD455294 ↗

使用两种不同节点的优劣势对比：

- LLM 节点
 - 优势：
 - Token 效率最高：无平台额外封装，检索结果直接以纯文本注入提示词，适合 Token 预算

有限的场景（如长文档检索 + 生成）；

- 灵活性拉满：可通过提示词控制“何时检索”（如简单问题直接回答，复杂问题触发检索）、“如何整合文档”（如按相关性排序后只取前 3 条核心信息），甚至实现自定义多轮检索；
- 可外接自定义检索工具：若平台检索功能不足，可通过提示词调用外接检索工具（如自建向量数据库），突破平台功能限制。

○ 劣势：

- 检索不可控：无法确认 LLM 是否真的使用了检索结果（可能忽略检索文档直接生成），也无法排查检索漏检、误检问题；
- 调试成本高：需要反复优化提示词（如明确指令“必须基于以下检索到的文档回答，不编造信息”），才能让 LLM 正确联动检索结果；
- 不支持检索指标评估：无法单独统计 Recall@k、MRR 等检索指标，只能通过生成答案的准确率间接判断。

● RAG 节点

○ 优势：

- 检索可观测、可评估：独立输出检索日志，支持查看召回文档、排序结果，可直接对接评估指标（如之前提到的 Hit@k、MRR），优化检索有明确数据支撑；
- 工程效率高：开箱即用，无需手动设计检索 – 生成链路，平台自动完成文档整理、提示词注入，快速落地 MVP；
- 稳定性强：平台封装了成熟的检索策略（如默认混合检索、重排），避免因提示词设计不当导致检索 – 生成脱节，生成答案的事实准确率更有保障。

○ 劣势：

- 少量额外 Token 消耗：如前所述，标准化封装会增加少量 Token，但通常不影响核心使用；
- 灵活性受限：高级检索逻辑（如自定义多轮检索、特殊融合规则）需依赖平台功能，若平台不支持则无法实现；
- 提示词定制空间小：平台提供的提示词模板多为固定格式，难以深度定制文档整合逻辑（如按业务逻辑筛选文档，而非仅按相关性）。

假如想兼具 LLM 节点的灵活性和 RAG 节点的可观测性，需要在 LLM 节点中明确要求生成检索结果，并且把检索结果输出用于统计。但是在当前阶段，我认为「RAG 检索的效果和可观测性」要优先于「少量额外 Token 的消耗」。

知识库检索要求：

****基础组件**** - 可查询 [SSCFE SSC UI React] 和 [SSCFE Pro Component] 知识库

****业务组件**** 只查询 [SSCFE Pro Component] 知识库

****图标**** 只查询 [Ssc Ui Icons Icons React] 知识库

****类型定义/props**** - 只查询 [Ssc Ui Icons Icons React] 知识库

参数：top_k=6, overlap_threshold=0.85。

输出：

仅输出最终 JSON 结构（不含说明文字）。

请为 JSON 结构添加 Retrieval 字段，返回多轮检索的内容，用于统计和量化检索效果。

User Message



用户的输入 开始 / query ABC 开始 / image

组件签名：

{{tools_llm_sthok06:1kav1s4}}

UI 知识库资料

[专业] Pro 组件库整体... / react_pro_component_func_list ABC

[基础] 基础组件库整体信息 / ssc_react_ui_func_list ABC

2.第二种情况，是自研 RAG，那么也要求 RAG 是个独立的，具备可观测性的功能点。例如 AI 辅助自测项目中，RAG 是在 MCP 中实现的，那么也需要把 RAG 这部分能力设计为一个可单独记录检索和生成的输入输出的能力。

评测集模板

接下来就是具体怎么进行评估了，这里提供了一份评测集模板，每一条用例包括了，输入，标准答案、输出三个关键部分，评估就是衡量输出和标准答案在不同指标上的差距的过程。评估的方法可以是人工对比、工程化对比、LLM 对比、Agent 对比等，这些过程是渐进的。

检索效果评测集模板

用例 ID	输入	标准答案	输出	检索 - 召回率	检索 - 准确率	其他指标	备注
示例 1: AI 生码 000001	列表查询	pro-table.html 编辑源码 pro-table 组件文档 本文档包含该组件的完整使用教程, 欢迎阅读或反馈, 帮助完善文档内容。 1. 组件描述 ProTable ProTable 组件提供了表格功能: - 表格渲染: 渲染了表格内容 - 数据过滤: 提供了过滤功能 - 数据排序: 提供了排序功能 - 数据导出: 提供了导出功能 - 数据删除: 提供了删除功能 - 数据新增: 提供了新增功能 - 数据编辑: 提供了编辑功能 - 数据批量操作: 提供了批量操作功能	<pre># ProTable\n## 2. 示例代码\n### 展示额外元素\n\n``tsx\n/*\n * title: 展示额外元素\n * desc:\nProTable 组件\n提供了 `slots`\n属性, 可以在\n搜索表单前\n后、操作按钮\n前后插入额外\n的元素.\n * background:\n'#f5f6f9'\n */\nimport *\nas React from\n'react';\n\nimport { Alert,\nButton } from\n'ssc-ui-react';\n\nimport { ProTable }\nfrom 'react-pro-components';\n\nimport type {</pre>	100%	100%		预期返回 ProTable 使用文档 实际返回 ProTable 使用文档 召回率和准确率都是 100%

			<pre>ProTableProps } from 'react- pro- components/t ypings/compo nents/pro- table/types';\n \nimport * as api from './api';\nimpor t type { ModelType, RecordType } from './mock- data- types';\n\nexp ort defa</pre>				
示例 2: 测 试用例 00000 2	列 表 查 询	列表搜索 – 单一筛选 列表搜索 – 多筛选 列表搜索 – 关联筛选 列表搜索 – 重置筛选 列表搜索 – 分页pageno 搜索 列表搜索 – 分页count搜 索	列表搜索 – 单 一筛选 列表搜索 – 多 筛选 列表搜索 – 关 联筛选 列表搜索 – 重 置筛选	6 7 %	1 0 0 %		预期返回 6 个列 表页用例 实际返回 4 个, 没有错误返回的 召回率 = 4/6 准确率 100%

用例 ID	输入	标准答案	输出	生成 - 准确性	生成 - 完整性	其他指标	备注
示例 1: AI 生成代码 0000 01	列表查询	仅部分示例	仅部分示例:	0 %	10 0 %		预期生成一个 Protable 的代码 实际生成了一个 table 的代码 准确性 0% 并且由于前面检索阶段准确率是 100%，那么就是生成阶段有问题。

```

1  const proFormProps: ProFormProps = {
2    ...
3  };
4  const proTableProps: ProTableProps = {
5    ...
6  };
7  return (
8    <div className={styles.page}>
9      <Layout.Blocks
10        direction="vertical"
11        style={{ backgroundColor: 'rgba(155,155,155,0.19)' }}
12      >
13        <PageHeader {...pageHeaderProps} />
14
15        <Layout.Blocks
16          direction="horizontal"
17          style={{ margin: '0px', padding: '0px' }}
18        >
19          <ProForm {...p

```

```

1  return (
2    <div className="table-demo-table-demo-quick-intro">
3      <Table
4        bordered={bordered}
5        dataSource={noData ? [] : dataSource}
6        columns={columns}
7        rowKey="id"
8        scroll={
9          {
10            x: 1200,
11            y: fixedHeight ? 360 : undefined,
12          }
13        }
14        sticky={
15          stickyHeader ? {
16            offsetHeader: 0, // 表头吸顶
17            offsetScroll: (showF

```

		<pre>20 roFormProps} /> <P roTable {... proTableProp 21 s} /> </La 22 yout.Blocks> </Layo 23 ut.Blocks> 24 </div> 25); };</pre>	<pre> ooter ? 58 : 0) + 4, // 横向滚 18 动条吸底 } : false 19 } 20 getS crollConta iner={() = > document .querySele ctor('.tab le-demo')! } 21 show Header={sh owHeader} 22 foot er={showFo oter ? () => '这是表 尾' : unde 23 fined} load 24 ing={loadi ng} onRe nderTimeRe port={onRe nderTimeRe port} 25 /> 26 </div> 27);</pre>			
示例 2: 测 试用 例 0000 02	列 表 查 询	▼ 1 ## 二、搜索功能测试 2 3 ### 2.1 单字 段搜索 - Outb ound Task ID 4 **测试ID**: T C_SEARCH_001	▼ 1 // 搜索功能 测试 2 ▼ search: 3 { 4 title: 5 "搜索功能测 试", 6 cases: 7 [8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100	10 0 %	50 %	生成的用例都是 对的，但是漏了 一些没生成，并 且没有按照要求 的格式输出，合 规率 0%

5	**优先级**: P	5	{
6	0	6	id
7	**测试步骤**:	7	: "TC003",
	1. 在"Outbound Task ID"输入框输入: `OUT-20250115-001`	8	name: "单条件搜索 - 出库任务ID",
8	2. 点击搜索按钮	9	description:
9			"使用出库任务ID进行精确搜索",
10	**预期结果**:	10	steps: [
11	- ✓ 触发`fetchData.fetcher`	11	"1. 在出库任务ID字段输入'AT0123456789'",
12	- ✓ 请求参数包含: `task_id: "OUT-20250115-001"`	12	"2. 点击搜索按钮",
13	- ✓ 重置分页到第1页	13	"3. 验证搜索结果"
14	- ✓ 显示匹配的搜索结果	14	],
15		15	expected: [
16	**Mock数据**:	16	"API调用参数包含task_id: 'AT0123456789'",
17	`mockSearchByTaskId`	17	"返回匹配的任务数据",
18	----	18	"表格显示搜索结果"
19		19	],
20	### 2.2 单字段搜索 - Inbound Station	20	mockData: "mockSearchByTaskId"
21	**测试ID**: TC_SEARCH_002	21	{
22		22	id
23	**优先级**: P		: "TC004",
24	0		
25	**测试步骤**:		
	1. 在"Inbound Station"下拉框选择: `[101]Bangkok Hub`		
26	2. 点击搜索按钮		
27	**预期结果**:		

```

28 - ✓ 请求参数包
    含: `inbound_
    station_id:
29 "101"`
    - ✓ 返回该站点
    的所有出库任务
30
31 **Mock数据**:
    `mockSearch
    ByStation`
32
33 ----
34
35 ### 2.3 单字
    段搜索 - SKU
    ID
36 **测试ID**: T
    C_SEARCH_003
37
38 **优先级**: P
    0
39 **测试步骤**:
    1. 在"SKU ID"
    输入框输入: `S
    KU-12345`
40 2. 点击搜索按
    钮
41
42 **预期结果**:
43 - ✓ 请求参数包
    含: `sku_id:
    "SKU-12345"`
44 - ✓ 返回包含该
    SKU的所有任务
45
46 **Mock数据**:
    `mockSearch
    BySkuId`
47
48 ----
49
50 ### 2.4 单字
    段搜索 - Deli
    very Method
51 **测试ID**: T
    C_SEARCH_004
52

```

```

23     na
    me: "单条件
    搜索 - 转移
    任务ID",
24     de
    scription:
    "使用ATT开
    头的转移任务
    ID搜索",
25     st
    eps: [
26     "1. 在出库
    任务ID字段输
    入'ATT9876
    54321'",
27     "2. 点击搜
    索按钮",
28     "3. 验证搜
    索参数转换"
    ],
29     ex
    pected: [
30     "API调用参
    数包含trans
    fer_task_i
    d: 'ATT987
    654321'",
31     "正确处理AT
    T前缀的任务I
    D"
    ],
32     mo
    ckData: "m
    ockSearchB
    yTransferT
    askId"
    },
33     {
34     id
    : "TC005",
35     na
    me: "多条件
    组合搜索",

```

```

53  **优先级**: P
54  1
55  **测试步骤**:
56  1. 在"Delivery Method"下拉
57  框选择: `Self
58  Pickup`
59  2. 点击搜索按钮
60
61  **预期结果**:
62  - ✓ 请求参数包
63  含: `delivery
    _method: 1`
64  - ✓ 返回对应配
    送方式的任務
65
66  **Mock数据**:
67  `mockSearch
68  ByDeliveryMe
69  thod`
70
71  ----
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

				54	na				
				55	me: "日期范围搜索",				
				56	description:				
					"使用预期出库日期范围进行搜索",				
				57	steps: [				
				58	"1. 选择预期出库日期范围: 2024-01-01 00:00:00 到 2024-01-31 23:59:59",				
				59	"2. 点击搜索按钮"				
				60	],				
				61	expected: [				
				62	"日期参数正确转换为时间戳",				
				63	"API调用包含expected_outbound_date参数"				
				64	],				
				65	mockData: "mockDateRangeSearch"				
				66	}				

评估流程： 构建评测集→执行检索 / 生成→计算指标→分析 → 迭代优化

评估能力能否平台化

// 待进一步思考，目前感觉有点难

问题解答

文章开头的几个问题，可以用以上的方法论解答：

1. 【AI 自测用例生成项目】：知识库里有相关的用例，但是检索的时候就是召不回怎么办？

- (1) 参考 RAG 优化策略，寻找可以优化召回的方案
- (2) 通过评估集，评估各个方案的召回率是否有优化，准确率是否有变差

2. 【PRD2Code 项目】：如果生码效果差怎么区分是物料的问题还是生成的问题？

把 RAG 的效果评估分成检索部分和生成部分。检索部分可以进一步分为数据预处理和检索。如果检索的评估分数低，可以认为是物料问题，如果生成的分数低，可以认为是生码问题。

3. 【PRD2Code 项目】：为什么我总结的 RAG 优化文档总是被挑战？

没有可量化的评估结果，例如：

总结对比

	token 数	准确度	噪音	稳定性	耗时（通过 agent tool 的方式）
expert	平均要高，噪音 chunk 和重复 chunk（出错的情况下超高）	中低（普遍有噪音，并且完整性不够，被截断）	高	低，有概率失败，并且是连续失败（我自己在知识库 playground 中也有遇到过）	22s
retrieval	中（token 开销稳定性高）基础组件低、pro 组件高	基础组件（高），业务组件（中高）	低	高，目前没遇到失败的情况	24s

总结：

背后都是同一个基础组件库（我检查了知识库的内容，都是组件库的文档）

Smart 只支持 Retrieval，在参数调整到 strong match 以及 match score 调到 0.95 的情况下，对于 button 组件的返回仍然有噪音；

alpha 的 Retrieval 在默认参数的情况下，检索的准确性比 smart 是更好的；

附录：学习资料

一文了解RAG18种常见算法对比，看这一篇就够了！

一文详谈RAG优化方案与实践

[《优化 RAG 系统的最佳实践与深度解析》](#)